

Hands-On-Learning: Modularize Intent Classification: Refactor 2000-Line Chatbot with LangChain

Description

Transform a monolithic chatbot codebase into a modular LangChain application by replacing hardcoded prompts with PromptTemplates, implementing model abstractions, and adding structured output parsing with proper error handling.

Learning Objectives

By the end of this activity, learners will be able to:

- Identify architectural problems in monolithic LLM code and map them to LangChain components
- Replace hardcoded prompts with reusable PromptTemplate objects enabling external configuration



- Implement model abstractions that enable provider-agnostic LLM integration
- Transform brittle string parsing into validated OutputParser implementations



Assignment Components

Introductory Slide:

→ **Title: Junior Developer Mission: Save TechSupport Inc's Failing Chatbot**

→ **Subtitle:** One week to prove LangChain can transform 2000 lines of spaghetti code into production-ready architecture

Tools to be used:

- LangChain (Python framework)
- OpenAI API or Claude API (LLM provider)
- Python 3.8+
- Git for version control
- Text editor/IDE (VS Code recommended)

Scenario

You've been hired as a Junior LLM Developer by TechSupport Inc, a B2B SaaS company providing customer service automation to over 500 enterprises. Their flagship chatbot handles 100,000+ daily queries but suffers from frequent crashes, security vulnerabilities from prompt injection, and maintenance nightmares. The current codebase has 2,000 lines with hardcoded prompts scattered across 23 functions, raw OpenAI API calls mixed with business logic, and string parsing with regex that breaks on edge cases. The engineering team has given you one week to prove that LangChain can solve these issues. Your task: refactor the most critical

component—the intent classification and response generation module—using LangChain's core components. Document your approach and demonstrate measurable improvements in code maintainability (target: 70% reduction in lines of code) and system reliability (target: eliminate crash scenarios).

Instructions for learners

Step 1: Audit the Legacy Code

- Download the legacy chatbot codebase from the provided repositor
- Review `intent_classifier.py` and identify hardcoded prompts, direct API calls, and string parsing logic
- Document specific problems: count prompts, API calls, error handling gaps
- Note the total lines of code in the original implementation

Step 2: Design LangChain Architecture

- Map identified problems to LangChain components (PromptTemplate, LLM wrapper, OutputParser)
- Design your PromptTemplate structure with appropriate variables
- Define OutputParser schema using Pydantic models for structured data
- Plan your configuration management (environment variables, external config files)

Step 3: Implement LangChain Components

- Install LangChain: `pip install langchain openai`
- Create PromptTemplate objects extracting hardcoded prompts into reusable templates
- Implement LLM wrapper (ChatOpenAI or ChatAnthropic) with proper configuration
- Build OutputParser using PydanticOutputParser with validation schema
- Connect components into LLMChain with proper error handling
- Move API keys to environment variables and create YAML/JSON config for prompts

Step 4: Test and Measure Improvements

- Run your refactored code with sample customer queries
- Count lines of code in new implementation (target: 70%+ reduction)
- Test edge cases that broke the original regex parsing
- Capture screenshots showing successful execution and structured output

Step 5: Document Your Refactoring

- Create before/after comparison showing code reduction
- Document component mappings (what problem → which LangChain component)
- Prepare your repository with clear directory structure and README

Deliverable for Submission (AI Grading)

1 Deliverable 1: Refactored Code Repository

Question prompt: What LangChain components (PromptTemplate, LLM wrapper, OutputParser) did you implement to replace the legacy chatbot's hardcoded prompts and brittle parsing, and how does your modular architecture improve maintainability compared to the original 2,000-line implementation?

Response Type: File Upload

File Types: Coding files (.py, .json, .yaml, .yml, .txt), Compressed archives (.zip)

2 Deliverable 2: Configuration Management System

Question prompt: How did you externalize configuration (API keys, model parameters, prompts) from your code, and why does this separation of concerns improve security and enable environment-specific deployments (dev/staging/prod) without code changes?

Response Type: File Upload

File Types: Text (.txt, .env), Coding files (.json, .yaml, .yml)

3 Deliverable 3: Performance Comparison Report

Question Prompt: What measurable improvements (code reduction percentage, prompts eliminated, error handling enhancements) did your LangChain refactoring achieve, and how do these metrics demonstrate that your solution solves TechSupport Inc's specific maintenance and scalability problems?

Response Type: File Upload or Rich Text

File Types: Text (.txt, .pdf, .docx), Image (.png, .jpg), Video (.mp4, .mov - max 15 min)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.